

Métodos Computacionais em Física de Altas Energias

C++

Marco Dias

22 de agosto de 2026

Programação Orientada a Objetos (OO):

- Estilo de programação que captura o comportamento do mundo real
- Oculta detalhes de implementação
- Permite pensar em termos do domínio do problema

Objetivos do C++:

- 1 Compatibilidade com C
- 2 Extensão do C com construções OO

FAQ: <http://www.parashift.com/c++-faq-lite/>

Características:

- Visão centrada em dados
- Dados e comportamento fortemente ligados
- Classes e objetos
- Estruturas de dados abstratas

Estrutura Básica do C++

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4 int main() {
5     cout << "Hello World!" << endl;
6     return 0;
7 }
```

Namespace:

- Fornece um escopo para evitar conflitos de nomes
- namespace std; reservado para bibliotecas padrão
- using permite usar identificadores sem qualificação

Construtores e Destrutores:

- Construtor: inicializa um objeto da classe
- Destrutor: finaliza um objeto da classe (chamado quando sai do escopo)

Tipos Nativos

Tipo	Descrição
bool	Booleano (true/false)
char	Caractere
int	Inteiro
float	Ponto flutuante (precisão simples)
double	Ponto flutuante (precisão dupla)
void	Sem valor
wchar_t	Caractere largo

Modificadores:

- short, long - tamanho
- signed, unsigned - sinal

Declaração e Definição:

```
1 int i;           // Declaração e definição
2 int i = 0;       // Com inicialização
3 int i = 0, j;     // Múltiplas variáveis
4 extern int i;     // Apenas declaração
```

Conversões de Tipo:

- Promoção (widening) - segura (automática pelo compilador)
- Demotação (narrowing) - pode perder informação

```
int inteiro = static_cast<double>(i)
```

Operadores Aritméticos:

```
1 + - * / % // Divisão inteira
2 pow(x, y) // Potência (math.h)
```

Atenção à divisão:

```
1 int a = 3 / 2; // a = 1 (divisão inteira)
2 double b = 3.0 / 2; // b = 1.5
3 double c = 3 / 2; // c = 1.0 (3/2 = 1, convertido para double)
```

Operadores de Atribuição:

```
1 = += -= *= /= %=
```

Operadores e Expressões

Operadores de Incremento/Decremento:

```
1 ++i    // Pré-incremento
2 i++    // Pós-incremento
3 --i    // Pré-decremento
4 i--    // Pós-decremento
```

Operadores Relacionais e Lógicos:

```
1 ==      !=      <      >      <=     >=
2 &&      ||      !
```

1	x	y	x && y	x y
2	F	F	F	F
3	T	F	F	T
4	F	T	F	T
5	T	T	T	T

Sintaxe:

```
1 tipo_retorno nome_funcao(parametros) {  
2     // corpo da função  
3     return valor;  
4 }
```

Exemplos:

```
1 // Função sem retorno  
2 void myFunction(int i) {  
3     cout << "Valor: " << i << endl;  
4 }  
5  
6 // Função com retorno  
7 int soma(int a, int b) {  
8     return a + b;  
9 }
```

Argumentos Padrão:

```
1 void myFunction(int i, int j = 1) {  
2     // j tem valor padrão 1  
3 }
```

Sobrecarga de Funções:

```
1 void myFunction(int i) { ... }  
2 void myFunction(double d) { ... }  
3 void myFunction(int i, double d) { ... }
```

Ponteiros

Ponteiros são usados para referenciar variáveis e endereços de memória.

Declaração:

```
1 int* p;           // p é um ponteiro para int
2 int* p = &i;      // p aponta para i
```

Operadores:

- & - operador de endereço
- * - operador de dereferência (indireção)

Exemplos:

```
1 int i = 5;
2 int* p = &i;      // p contém o endereço de i
3 int j = *p;       // j = 5 (valor apontado por p)
4 *p = 10;          // i = 10
```

Ponteiro Nulo:

```
1 int* p = nullptr; // C++11
2 int* p = NULL;    // C++98
3 int* p = 0;        // Valor 0
```

Aritmética de Ponteiros:

```
1 int a[5] = {1, 2, 3, 4, 5};
2 int* p = a;           // p aponta para a[0]
3 p++;                  // p aponta para a[1]
4 cout << *p;           // Imprime 2
```

Arrays

Declaração:

```
1 int a[tamanho];           // Array unidimensional
2 int a[3] = {1, 2, 3};     // Com inicialização
3 int a[] = {1, 2, 3};      // Tamanho inferido
```

Acesso:

```
1 a[0] = 1;
2 a[1] = 2;
3 a[2] = 3;
4 int x = a[0];             // x = 1
```

Arrays Multidimensionais:

```
1 int a[2][2];              // Matriz 2x2
2 int b[2][5][200];         // 3D
```

Arrays como Ponteiros:

```
1 int a[5] = {1, 2, 3, 4, 5};  
2 int* p = a;           // p aponta para a[0]  
3 int* p = &a[0];       // Equivalente
```

Limitações:

- Tamanho fixo em tempo de compilação
- Sem verificação de limites
- Não podem ser copiados diretamente

Estruturas (struct)

Definição:

```
1 struct complex {  
2     double re, im;  
3 };  
4 complex c;  
5 c.re = 1.0;  
6 c.im = 2.0;
```

Estruturas com Funções Membros:

```
1 struct complex {  
2     void reset() {  
3         re = 0;  
4         im = 0;  
5     }  
6     double re, im;  
7 };  
8 c.reset();    // re = 0, im = 0
```

Controle de Acesso:

```
1 struct complex {  
2 public:  
3     void reset() { re = 0; im = 0; }  
4     double getRe() const { return re; }  
5 private:  
6     double re, im;    // Ocultos  
7 };
```

Classes são introduzidas pela palavra-chave `class`. Por padrão, os membros são `private`.

Definição:

```
1 class complex {  
2 public:  
3     void reset() {  
4         re = 0;  
5         im = 0;  
6     }  
7 private:  
8     double re, im;  
9 };
```

Operador de Resolução de Escopo (::):

```
1 ::i           // Escopo externo
2 complex::i    // Escopo da classe
3 complex::Y::i // Classe aninhada
```

Classes Aninhadas:

```
1 class X {
2     int i;           // X::i
3 public:
4     class Y {
5         int i;       // X::Y::i
6     };
7 };
```

Membros Estáticos:

- Compartilhados por todas as variáveis da classe
- Armazenados em um único local

```
1 class X {  
2 public:  
3     static int count();    // Função estática  
4     static int total;      // Variável estática  
5 };  
6  
7 int X::total = 0;          // Definição
```

Membros Const:

- Não podem modificar membros de dados
- Const-correctness torna o código mais robusto

```
1 class X {  
2 public:  
3     int getValue() const;    // Não modifica membros  
4     void setValue(int v);    // Pode modificar  
5 };  
6  
7 int X::getValue() const {  
8     return value;           // OK  
9     // value = 5;           // ERRO!  
10 }
```

Saída (ostream):

```
1 #include <iostream>
2 std::cout << "Hello World!" << std::endl;
3 std::cout << "Número: " << 42 << std::endl;
```

Entrada (istream):

```
1 int i;
2 std::cin >> i;
3 std::string s;
4 std::cin >> s;
```

Arquivos (fstream):

```
1 #include <fstream>
2 // Leitura
3 ifstream inFile("arquivo.txt", ios::in);
4 int var;
5 inFile >> var;
6 // Escrita
7 ofstream outFile("saida.txt", ios::out);
8 outFile << "Dados" << endl;
```

Lendo linha por linha:

```
1 string linha;
2 while (getline(inFile, linha)) {
3     cout << linha << endl;
4 }
```

Exemplo: MET

Arquivo SimpleMET.h:

```
1 #ifndef SIMPLEMET_H
2 #define SIMPLEMET_H
3 #include "MyEvent.h"
4 class SimpleMET {
5 public:
6     SimpleMET();
7     ~SimpleMET();
8     void Add(MyTrack);
9     void Add(MyJet);
10    MyMET GetMET();
11    double Value();
12 private:
13     // Membros privados
14 };
15 #endif
```

Exemplo: MET

Uso:

```
1 SimpleMET met;  
2 met.Add(track);  
3 met.Add(jet);  
4 MyMET result = met.GetMET();  
5 double value = met.Value();
```

Construtores e Destrutores

Construtor: Inicializa objetos da classe. Mesmo nome da classe.

Destruidor: Destrói objetos. Nome precedido por ~.

```
1 class Example {
2 public:
3     Example();           // Construtor padrão
4     Example(int i);      // Construtor com parâmetro
5     ~Example();          // Destruidor
6 };
7
8 // Uso
9 Example e;               // Construtor padrão
10 Example* ep = new Example; // Novo objeto
11 delete ep;               // Destruidor
12 Example e(5);            // Construtor com parâmetro
```

Inicializador de Construtor:

```
1 Example::Example() : i(0) { // Inicializa membro i
2     // Corpo
3 }
```

Construtor Padrão:

- Não requer argumentos
- Inicializa arrays de objetos
- Se não definido, sistema fornece um

Construtor de Cópia

Chamado quando um novo objeto é criado a partir de outro objeto.

```
1 class Example {  
2 public:  
3     Example(const Example& other); // Construtor de cópia  
4 };  
5 // Uso  
6 Example e(0); // Construtor normal  
7 Example f(e); // Construtor de cópia  
8 Example f = e; // Construtor de cópia (inicialização)
```

Cópia Rasa (Shallow) vs Cópia Profunda (Deep):

- **Rasa:** Copia apenas os ponteiros (mudança afeta ambos)
- **Profunda:** Copia o conteúdo (memória separada)

```
1 i = e.i; // Cópia rasa (padrão)  
2 Example e,f; f=e; // Cópia profunda (implementação manual)
```

Regra dos Três: Se precisar de construtor de cópia, provavelmente também precisa de:

- 1 Destrutor
- 2 Operador de atribuição (operator=)

Mecanismo de derivar uma nova classe a partir de uma existente.

Sintaxe:

```
1 class derivada : public base {  
2     // Membros adicionais  
3 };
```

Modificadores de Acesso:

Palavra-chave	Significado
public	Membros públicos da base são públicos na derivada
protected	Membros públicos da base são protegidos na derivada
private	Membros públicos da base são privados na derivada

Virtual Functions:

- Funções declaradas na classe base
- Redefinidas na classe derivada
- Seleção em tempo de execução (polimorfismo)

Função Virtual Pura:

```
1 virtual <tipo> func_prototype() = 0; // Classe abstrata
```

Templates

Permitem usar o mesmo código com diferentes tipos.

Template de Classe:

```
1 template <class T>
2 class Stack {
3 private:
4     vector<T> data;
5 public:
6     void push(const T& value) {
7         data.push_back(value);
8     }
9     T pop() {
10         T value = data.back();
11         data.pop_back();
12         return value;
13     }
14 };
```

Templates

Uso

```
1 Stack<int> intStack;  
2 Stack<string> stringStack;
```

Template de Função:

```
1 template <typename T>  
2 T myMax(T x, T y) {  
3     return (x > y) ? x : y;  
4 }  
  
5  
6 // Uso  
7 cout << myMax<int>(3, 7) << endl;  
8 cout << myMax<double>(3.5, 2.1) << endl;
```

Containers Comuns:

Tipo	Descrição
vector	Array dinâmico
list	Lista duplamente encadeada
map	Dicionário (chave-valor)
set	Conjunto (sem duplicatas)
queue	Fila
stack	Pilha
pair	Par de valores

Famílias:

- **Sequência:** vector, list (ordenados)
- **Associativos:** set, map (chave para busca)

Iterador: Ponteiro para navegar sobre containers.

```
1 vector<int> intVec;  
2 intVec.push_back(123);  
3 intVec.push_back(456);  
4  
5 vector<int>::const_iterator i;  
6 for(i = intVec.begin(); i != intVec.end(); i++) {  
7     cout << *i << endl;  
8 }
```

Map:

```
1 #include <map>
2
3 map<string, int> cut;
4 cut["pt"] = 20;
5 cut["deltaPhi"] = 175;
6
7 if(track.pt() < cut["pt"]) continue;
8
9 // Busca
10 map<string, int>::const_iterator i = cut.find("pt");
11 if(i != cut.end()) {
12     cout << "pt = " << i->second << endl;
13 }
```

Pair:

```
1 #include <utility>
2
3 // Retornando par de valores
4 pair<JetTag, Coll> tag() {
5     JetTag tag = ...;
6     Coll c = ...;
7     return pair<JetTag, Coll>(tag, c);
8 }
9
10 // Uso
11 pair<JetTag, Coll> ip = tag();
12 JetTag tag = ip.first;
13 Coll c = ip.second;
```

Polimorfismo dá diferentes significados à mesma mensagem.

Sobrecarga de Funções:

```
1 class MyHistogram {  
2 public:  
3     void fill(string name, double value);  
4     void fill(string name, double value1, double value2);  
5 };  
6 // Uso  
7 hist.fill("pt", 45.5);  
8 hist.fill("eta_phi", 1.2, 2.3);
```

Sobrecarga de Operadores:

```
1 class Point {
2 public:
3     Point operator+(const Point& p) const {
4         Point result;
5         result.x = this->x + p.x;
6         result.y = this->y + p.y;
7         return result;
8     }
9 private:
10     double x, y;
11 };
12
13 // Uso
14 Point p1, p2;
15 Point p3 = p1 + p2;
```

Exemplo: Matrix * Vector

```
1 class Vector {  
2 public:  
3     double x, y, z;  
4     // ...  
5 };  
6 class Matrix {  
7 public:  
8     double m[3][3];  
9     // ...  
10};
```

Exemplo: Matrix * Vector

```
1 // Operador matrix * vector
2 Vector operator*(const Matrix& m, const Vector& v) {
3     Vector result;
4     result.x = m.m[0][0]*v.x + m.m[0][1]*v.y + m.m[0][2]*v.z;
5     result.y = m.m[1][0]*v.x + m.m[1][1]*v.y + m.m[1][2]*v.z;
6     result.z = m.m[2][0]*v.x + m.m[2][1]*v.y + m.m[2][2]*v.z;
7     return result;
8 }
9 // Uso
10 Matrix m;
11 Vector v;
12 Vector result = m * v;
```

Exemplo: Classe MyVertex

```
1 class MyGlobalPoint {  
2 public:  
3     MyGlobalPoint();  
4     ~MyGlobalPoint();  
5     double getX() const;  
6     double getY() const;  
7     double getZ() const;  
8     double value() const;  
9     double error() const;  
10    double getPhi() const;  
11    MyGlobalPoint operator+(const MyGlobalPoint&) const;  
12    MyGlobalPoint operator-(const MyGlobalPoint&) const;  
13 protected:  
14    double x, y, z;  
15    double dxx, dxy, dxz, dyy, dyz, dzz;  
16 };
```

Exemplo: Classe MyVertex

```
1 class MyVertex : public MyGlobalPoint {  
2 public:  
3     MyVertex();  
4     ~MyVertex();  
5     double Eta() const;  
6     double Phi() const;  
7     MyVertex operator+(const MyVertex&) const;  
8     MyVertex operator-(const MyVertex&) const;  
9     vector<MyTrack> tracks() const;  
10 private:  
11     vector<MyTrack> assocTracks;  
12 };
```

Arquivo MyRootTree.h:

```
1 #ifndef MYROOTTREE_H
2 #define MYROOTTREE_H
3 #include "TROOT.h"
4 #include "TFile.h"
5 #include "TTree.h"
6 #include "MyEvent.h"
7 class MyRootTree {
8 public:
9     MyRootTree();
10    ~MyRootTree();
11    void fillTree(MyEvent*);
12 private:
13     TTree* rootTree;
14     MyEvent* myEvent;
15     TFile* rootFile;
16 };#endif
```

ROOT: Implementação

```
1 #include "MyRootTree.h"
2 MyRootTree::MyRootTree() {
3     rootFile = new TFile("analysis.root", "RECREATE");
4     rootTree = new TTree("rootTree", "events");
5     myEvent = new MyEvent();
6     int bufsize = 256000;
7     int split = 1;
8     rootTree->Branch("MyEvent", "MyEvent", &myEvent, bufsize, split);
9 }
10 MyRootTree::~MyRootTree() {
11     rootFile->cd();
12     rootTree->Write();
13     rootTree->ls();
14     rootFile->Close();
15     delete rootFile;
16 }
17 void MyRootTree::fillTree(MyEvent* event) {
18     myEvent = event;
19     rootFile->cd();
20     rootTree->Fill();
21 }
```

ROOT: Uso no Analisador

```
1 class MyEventAnalyser : public edm::EDAnalyzer {
2 private:
3     MyRootTree* userRootTree;
4
5     void MyEventAnalyser::beginJob() {
6         userRootTree = new MyRootTree();
7     }
8
9     void MyEventAnalyser::analyze(const edm::Event& iEvent) {
10        // Reconstrução do evento
11        MyEvent* ev = new MyEvent();
12        ev->primaryVertex = PV;
13        ev->electrons = recElectrons;
14        ev->muons = recMuons;
15        ev->mcParticles = mcParts;
16        userRootTree->fillTree(ev);
17    }
18
19    void MyEventAnalyser::endJob() {
20        delete userRootTree;
21    }
22 };
```